

Quero Trabalhar

Relatório Técnico do Projeto

Separação de repositórios, containerização Docker e integração completa do frontend com a API

Setembro de 2026

Sumário executivo

Este documento consolida todo o trabalho realizado no projeto Quero Trabalhar: a separação do frontend e do backend em repositórios independentes, a criação e validação de toda a infraestrutura Docker (em container único e em containers separados), e a integração completa da interface React com a API real do backend — cobrindo todas as entidades disponíveis, sem dados simulados ('mock').

Ao longo do processo, **13 bugs reais** foram encontrados e corrigidos — a maioria só visível testando de verdade contra o backend rodando em container, não apenas lendo o código. Cada correção foi validada no navegador, com chamadas reais à API.

Item	Resultado
Repositórios separados	quero-trabalhar (backend) e quero-trabalhar-front (frontend)
Containers Docker	3 configurações: único, separado (local) e separado (branch compatível)
Telas novas criadas	Empresas, Recrutador, Indicações, Admin (4 sub-telas)
Telas reescritas	Perfil e Criar Vaga (continham campos sem correspondência na API)
Bugs reais corrigidos	13, detalhados nas seções seguintes
Cobertura da API	100% dos endpoints documentados possuem tela ou módulo correspondente

1. Contexto e decisões estruturais

1.1 Separação em dois repositórios

O projeto começou como um único repositório com o backend Spring Boot e uma pasta frontend/ dentro dele. Essa estrutura foi desfeita: o frontend passou a viver em seu próprio repositório, **quero-trabalhar-front**, e o backend permaneceu em **quero-trabalhar**. Cada um ganhou seu próprio Dockerfile, docker-compose e documentação.

1.2 A decisão mais importante: qual backend o frontend deveria seguir

O repositório do backend tem, além da branch principal, uma branch **jefferson-experimental** significativamente mais avançada — com módulos de Empresa, Localidade (país/estado/cidade), painel de administração, indicações, experiências profissionais e um contrato de API documentado em uma coleção Postman completa. A branch principal não tem nada disso: os endpoints são diferentes (/auth/login em vez de /login, por exemplo) e o token de autenticação vem no *corpo* da resposta, não no cabeçalho.

Diante dessa divergência, a decisão tomada — e mantida em todo o projeto — foi: **o frontend é escrito contra o contrato documentado (coleção Postman), que corresponde à branch jefferson-experimental**, sem nunca alterar as branches do repositório do backend. Sempre que um backend real precisou ser executado para testes, ele foi obtido clonando essa branch diretamente do GitHub durante o build da imagem Docker — nunca fazendo checkout local dela.

Consequência prática: rodar o frontend contra a branch principal do backend, hoje, resultaria em erros 404 na maior parte das telas. O par funcional é sempre frontend + jefferson-experimental.

2. Infraestrutura Docker

2.1 Container único (docker-allinone/)

Uma imagem que builda o frontend (Vite) e clona + compila o backend da branch jefferson-experimental dentro do mesmo `Dockerfile`, rodando os dois processos (Java na porta 8080, nginx na porta 80) lado a lado no mesmo container. Existe para testes rápidos, com um único `docker run`. Um script `entrypoint.sh` derruba o container inteiro se qualquer um dos dois processos morrer, evitando mascarar falhas.

2.2 Containers separados

Duas variações foram criadas, cada uma com front e backend em containers independentes, comunicando-se por uma rede Docker compartilhada (`quero-trabalhar-net`):

- **docker-compose.full.yml** — builda o backend a partir do checkout local do repositório `quero-trabalhar` (útil apenas se essa branch local já for compatível).
- **docker-compose.separated.yml** — builda o backend clonando `jefferson-experimental` do GitHub (via `docker-backend-jefferson/Dockerfile`), garantindo compatibilidade real com o frontend. É a configuração recomendada para testes completos.

2.3 Como front e backend se comunicam

O frontend nunca chama o backend diretamente do navegador. Em desenvolvimento, o servidor do Vite atua como proxy reverso; em produção/container, é o nginx que assume esse papel, encaminhando `/login`, `/api/**` e `/swagger-ui` para o backend. Isso elimina problemas de CORS e garante que o cabeçalho `Authorization` devolvido pelo login chegue intacto ao navegador.

2.4 Bugs de infraestrutura corrigidos

#	Bug	Causa e correção
1	Nginx não iniciava quando o backend ainda não estava de pé	proxy_pass com hostname literal resolve DNS ao carregar a config, travando o nginx se o backend não responder nesse instante. Corrigido com resolver do Docker (127.0.0.11) + variável no proxy_pass, resolvendo por requisição.
2	Healthcheck do container marcava 'unhealthy' mesmo com tudo funcionando	No Alpine (musl), o wget resolve 'localhost' preferindo IPv6 (::1), mas o nginx só escuta IPv4. Corrigido usando 127.0.0.1 explícito no healthcheck — replicado em três Dockerfiles diferentes.

3. Camada de API do frontend

Toda comunicação com o backend passa por `src/api/`, com um módulo por grupo de endpoints (auth, usuarios, oportunidades, empresas, recrutadores, candidatos, experiencias, indicacoes, tiposDeEmprego, localidades, admin). Nenhum componente monta URL manualmente.

O núcleo (`client.js`) cuida de: injetar o token JWT nas chamadas autenticadas; ler/gravar o token no `localStorage`; converter qualquer erro HTTP em uma classe `ApiError` com mensagem legível; descartar parâmetros de busca vazios; e normalizar a paginação do Spring (`Page<T>`) para um formato estável.

4. Telas do frontend

4.1 Telas existentes, ligadas à API real

- **Login** — antes existia mas nunca era exibida; App.jsx não tinha portão de autenticação. Agora é a porta de entrada obrigatória.
- **Cadastro** — recebeu os campos CPF e telefone, obrigatórios na API e ausentes no formulário original.
- **Vagas** — busca com debounce, paginação, candidatura real, estados de carregando/erro/vazio.

4.2 Telas reescritas do zero

Duas telas continham campos inventados, sem qualquer correspondência nas entidades do backend:

- **Perfil** — a versão anterior tinha CPF editável, escolaridade, endereço e 'sobre você', nenhum existente no backend. Reescrita como: dados da conta (nome/telefone/e-mail), troca de senha, ativação de perfis (candidato/recrutador), experiências profissionais e vagas de interesse — tudo real.
- **Criar Vaga** — tinha campos como 'remuneração', 'requisitos' e endereço livre, nenhum existente na API. Reescrita usando tipo de emprego (com sugestão inline de tipos novos), modalidade e localidade via autocomplete país → estado → cidade.

4.3 Telas novas, criadas nesta integração

Tela	Funcionalidade coberta
Empresas	Listar, buscar, criar, ver detalhe (recrutadores e vagas vinculadas)
Recrutador	Perfil do recrutador, vínculo com empresa, gestão das próprias vagas (criar/editar/excluir)
Indicações	Criar, listar (dadas e recebidas), remover
Admin → Usuários	Listar, editar dados, ativar/remover perfis, remover usuário
Admin → Tipos de Emprego	Aprovar sugestões (individual e em lote), criar direto no catálogo, remover
Admin → Experiências	Listar todas, editar, remover
Admin → Vínculos de Recrutador	Aprovar/recusar vínculo com empresa

4.4 Componentes compartilhados criados

Para evitar duplicar lógica entre as novas telas: `shared/Estado.jsx` (mensagens de aviso/erro, paginação), `shared/Painel.jsx` (seções, badges, cards de item), `shared/SeletoresLocalidade.jsx` (autocomplete país→estado→cidade) e `shared/SeletoresTipoDeEmprego.jsx` (seleção com sugestão de novo tipo).

5. Bugs reais encontrados e corrigidos

Lista completa, em ordem cronológica. Os primeiros seis foram encontrados por auditoria de código (comparando o contrato documentado com o código-fonte atual do backend); os demais só apareceram testando de verdade no navegador contra o backend em execução.

Bug 1–4 — 4 endpoints usavam POST em vez de PUT

adicionarPerfilCandidato e adicionarPerfilRecrutador (usuário e admin) chamavam POST; o controller atual usa PUT. Corrigido nos 4 pontos.

Bug 5 — Endpoint de atualizar tipo de emprego (admin) não existe

buscarTipoDeEmprego e atualizarTipoDeEmprego chamavam GET/PUT em `/api/admin/tipos-emprego/{id}`, que não existe no controller atual. Funções removidas.

Bug 6 — Endpoint de atualizar a própria experiência não existe

experiencias.atualizar chamava um PUT que só existe no fluxo de admin, não no do próprio candidato. Função removida; editar hoje é remover e recriar.

Bug 7 — `/api/localidades/*` exige termo de busca com 2+ caracteres

Não é uma listagem completa, é autocomplete. O seletor de localidade foi reescrito de um estático para um campo de busca de verdade.

Bug 8 — Autocomplete de localidade reabria o dropdown sozinho após selecionar

Selecionar uma opção muda o texto para o nome escolhido, que tem 2+ letras, disparando nova busca automaticamente. Corrigido comparando o termo digitado contra o último valor efetivamente selecionado — uma abordagem por 'flag de uso único' havia falhado por causa do Strict Mode do React, que executa efeitos duas vezes em desenvolvimento.

Bug 9 — Busca com debounce travava em 'Carregando...' para sempre

Em Vagas, Empresas e Admin de Usuários: o efeito de debounce reativava o estado de carregamento 400ms após qualquer montagem, mesmo sem digitação, e nada o desligava de volta porque o termo de busca não mudava de valor de fato. Corrigido comparando o termo digitado contra o já aplicado antes de disparar uma nova busca.

Bug 10 — Aprovar tipo de emprego individualmente dava 'Token inválido'

Mensagem enganosa: o endpoint exige um corpo `{titulo, descricao}` que não estava sendo enviado. O erro de parsing do Spring era redespachado internamente de um jeito que produzia esse 401 confuso, sem relação real com o token.

Bug 11 — Container do frontend marcava 'unhealthy' (bug de infraestrutura)

Ver seção 2.4 — corrigido em três Dockerfiles diferentes.

Bug 12 — Nginx não subia se o backend ainda não estivesse pronto (bug de infraestrutura)

Ver seção 2.4.

Bug 13 — Erro de validação de CPF exibia o texto bruto de uma exceção Java

O backend não formata corretamente falhas de validação que ocorrem na entidade (em vez do DTO) — vaza o `toString()` de um `ConstraintViolationException`. Como não é possível corrigir esse código (pertence a outra branch), o frontend passou a reconhecer esse formato e extrair uma mensagem legível ('CPF inválido — confira os números digitados.').

6. Limitações reais da API (não são falhas do frontend)

Três pontos onde a interface é honesta sobre uma limitação do backend, em vez de simular uma funcionalidade que não existe:

- **Áreas de interesse do candidato** — existe uma entidade Preferencia no backend (tipos de emprego preferidos, disposição para trabalho remoto, localidades de interesse), mas nenhum endpoint a expõe. Não há como implementar essa funcionalidade até o backend criar a rota correspondente.
- **Busca de usuário por nome ao criar uma indicação** — o formulário de indicação exige o ID numérico do usuário porque não existe endpoint público de busca por nome ou e-mail (só o admin consegue listar usuários).
- **Lista de vínculos de recrutador pendentes** — o painel de admin permite aprovar ou recusar um vínculo, mas não há endpoint que liste quais estão pendentes; o administrador precisa informar o ID do recrutador manualmente.

7. Como rodar o ambiente de teste

Rede Docker compartilhada (uma vez só):

```
docker network create quero-trabalhar-net
```

Subir os dois containers a partir do repositório do frontend:

```
docker compose -f docker-compose.separated.yml up -d --build
```

Frontend em `http://localhost:5173`, backend em `http://localhost:8080` (Swagger em `/swagger-ui.html`).

7.1 Contas de teste

O perfil 'local' do backend popula o banco H2 (em memória) automaticamente a cada início. Senha **123456** para todas as contas — dado de teste, não credencial de produção.

E-mail	Perfil	Observação
admin@gmail.com	Administrador + Recrutador	Única conta com acesso ao painel de Admin; também pode criar vagas (empresa 'Admin Corp')
rh@tech.com	Recrutador	Bob Red, empresa 'Tech Solutions'
jane@gmail.com	Candidato + Recrutador	Perfil duplo, empresa 'Autonomo'
dan@gmail.com	Candidato	Dan Pink
emma@gmail.com	Candidato	Emma Black

Documento gerado automaticamente a partir do histórico de desenvolvimento do projeto Quero Trabalhar. Reflete o estado do trabalho até a data de geração.